Assignment 5:

Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def calculate_similarity(sent1, sent2):
     vectorizer = TfidfVectorizer()
     tfidf_matrix = vectorizer.fit_transform([sent1, sent2])
     sim_matrix = cosine_similarity(tfidf_matrix[0:1],
tfidf_matrix[1:2])
     similarity_score = sim_matrix[0][0]
     return similarity_score

sentence1 = "This is a sample sentence."
sentence2 = "This sentence is just a sample."

score = calculate_similarity(sentence1, sentence2)
rounded_score = round(score, 1)
print("Similarity score (rounded):", rounded_score)
```

```
>>> print("Similarity score (rounded):", rounded_score)
Similarity score (rounded): 0.8
```

Assignment 6

Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# Define the dataset
data = {'text': ['This is a sample document.', 'Another document
with different content.']}
documents = data['text']

# Create a TF-IDF Vectorizer instance and compute the TF-IDF
matrix.
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)

# Convert the matrix to a DataFrame for a clearer view.
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(),
columns=vectorizer.get_feature_names_out())
```

```
print("TF-IDF Matrix:")
print(df_tfidf)
```

```
>>> print(df_tfidf)
    another   content  different  document        is    sample      this      with
0  0.000000  0.000000   0.000000  0.379978  0.534046  0.534046  0.534046  0.000000
1  0.471078  0.471078   0.471078  0.335176  0.000000  0.000000  0.000000  0.471078
>>>
```

Assignment 7

Code:

```
import re
import nltk
from nltk.corpus import stopwords

# Ensure the necessary NLTK resources are downloaded.
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')

def preprocess_text(text):
      # Convert to lowercase.
      text = text.lower()

      # Remove punctuation using regex.
      text = re.sub(r'[^\w\s]', '', text)

      # Tokenize the text.
      tokens = nltk.word_tokenize(text)

      # Remove stopwords.
      stop_words = set(stopwords.words('english'))
      filtered_tokens = [token for token in tokens if token not in
stop_words]

      # Return the cleaned text.
      return ' '.join(filtered_tokens)

# Input text
input_text = "This is a sample text. It contains punctuation and
stopwords."

# Preprocess the text.
output_text = preprocess_text(input_text)
print("Processed text:", output_text)
```

```
>>> print("Processed text:", output_text)
Processed text: sample text contains punctuation stopwords
```

Assignment 8

Code:

```
import nltk
from nltk import pos_tag, ne_chunk, word_tokenize

# Download necessary NLTK resources.
nltk.download('punkt')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
def extract_named_entities(text):
      # Tokenize and tag the text.
      tokens = word_tokenize(text)
      pos_tags = pos_tag(tokens)

      # Perform Named Entity Recognition.
      tree = ne_chunk(pos_tags, binary=False)

      entities = []
      current_entity = []
      current_label = None

      for node in tree:
      # If the node is a named entity subtree.
      if hasattr(node, 'label'):
            entity_name = " ".join([leaf[0] for leaf in
node.leaves()])
            entity_type = node.label()
            # We collect only PERSON and ORGANIZATION entities.
            if entity_type in ['PERSON', 'ORGANIZATION']:
                  entities.append((entity_name, entity_type))
      # Else ignore non-entity tokens.
      return entities

# Input sentence
sentence = "John Smith works at Google."

# Extract named entities.
named_entities = extract_named_entities(sentence)
print("Named Entities:", named_entities)
```

```
>>> print("Named Entities:", named_entities)
Named Entities: [('John', 'PERSON'), ('Smith', 'PERSON'), ('Google', 'ORGANIZATION')]
>>>
```